

CHAVALES ENGINE

Guía de Estilo

Andrés García, Ismael Ortega, Cynthia Tristán, Pablo Iglesias, Iván Palomino, Sergio Naranjo, Jule Page, Nieves Alonso, Daniel Ramos y Javier Gómez

CHAVALES

0. Motivación

Documento normativo y conciso para mantener las convenciones de estilo a lo largo de todo el motor de manera fácilmente consultable en pro de la legibilidad, mantenibilidad y usabilidad del código en C++.

1. Idioma.....	1
2. Comentarios.....	2
3. Nombres.....	2
3.1. Clases, estructuras y enumerados:.....	2
3.2. Funciones:.....	2
3.3. Variables:.....	2
3.4. Constantes:.....	3
3.5. Claves de recursos:.....	3
3.6. Espacios de nombres:.....	3
4. Indentaciones.....	3
5. Directivas de compilación.....	4
6. General.....	4
7. Referencias.....	4

1. Idioma

La lengua usada para los comentarios será el español para mejor entendimiento, para todo lo demás será el inglés para mejor estándar.

- **Comentarios:** español
- **Resto:** inglés

2. Comentarios

- 2.1. **Formato:** estándar C++/doxygen
- 2.2. **Ejemplo:**

```

/**
 * @brief Frase corta descriptiva del método (opcional)
 *
 * Descripción detallada
 *
 * @param argumento - Descripción del parámetro
 * @note Nota aclaratoria (opcional)
 * @warning Aviso (opcional)
 * @return descripción del retorno
 */
void* funcion(int argumento) { ... }

```

3. Nombres

3.1. Clases, estructuras y enumerados:

3.1.1. **Formato:** PascalCase

3.1.2. **Tipo de palabra(s):** sustantivos

3.1.3. **Ejemplo:**

```

class ExampleComponent;
struct DataPackage;
enum RenderFlags;

```

3.2. Funciones:

3.2.1. **Formato:** camelCase, barra baja (_) como prefijo si son privadas

3.2.2. **Tipo de palabra(s):** verbos o frases verbales

3.2.3. **Ejemplo:**

```

void update(float deltaTime) { ... }
int _sumar(int a, int b) { ... }

```

3.3. Variables:

3.3.1. **Formato:** camelCase, barra baja (_) como prefijo si son privadas

3.3.2. **Tipo de palabra(s):** nombres descriptivos

- sin abreviaturas

3.3.3. **Ejemplo:**

```

public:
    Transform transform;
private:
    bool _isRed

```

3.4. Constantes:

3.4.1. **Formato:** SCREAMING_SNAKE_CASE

3.4.2. Tipo de palabra(s): sustantivos

3.4.3. Ejemplo:

```
const int MAX_ENTITIES = 10000;  
constexpr float MAX_DELTA = 0.33f;
```

3.5. Claves de recursos:

3.5.1. Formato: SCREAMING_SNAKE_CASE

3.5.2. Tipo de palabra(s): sustantivos descritos

3.5.3. Ejemplo:

```
loadTexture("RICE")
```

3.6. Espacios de nombres:

3.6.1. Formato: lowercase

- una única palabra
- posible abreviación

3.6.2. Tipo de palabra(s): sustantivos

3.6.3. Ejemplo:

```
namespace engine {  
    namespace ecs {  
        class Test...  
    }  
}  
engine::ecs::Test
```

4. Indentaciones

4.1. Formato: *Allman / K&R*

4.2. Ejemplo:

```
int foo(int a, int b) // indentación Allman  
{  
    a += b  
    return a  
}
```

```
int foo(int a, int b) { // indentación K&R  
    a += b  
    return a  
}
```

5. Directivas de compilación

#pragma once por definición general, exceptuando casos en los que un mismo archivo pueda ser incluido por múltiples rutas o sea dependiente de reinclusión en los que se usará #ifndef #define #endif

6. General

- 6.1. Indentaciones siempre por tabulaciones.
- 6.2. Separar operadores del resto de la sentencia por un espacio.
- 6.3. Uso explícito de los namespaces sin declaración “using”.
- 6.4. **Ejemplo:**

```
int main()
{
    std::cout << "Hello World!\n"
}
```

7. Referencias

- [cppreference - Identifiers](#)
- [cpp-coding-guidelines](#)